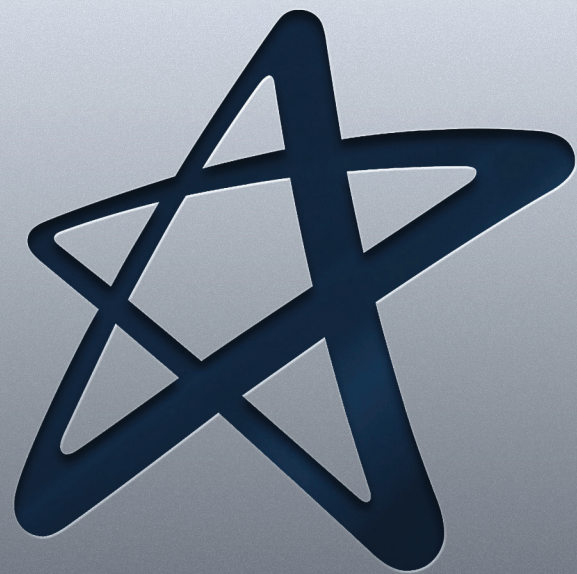


Compiladores e Interpretadores



Cruzeiro do Sul Virtual
Educação a distância

Material Teórico



Análise Sintática

Responsável pelo Conteúdo:

Prof. Dr. Cleber Silva Ferreira da Luz

Revisão Textual:

Prof.^a Dr.^a Luciene Oliveira da Costa Granadeiro

UNIDADE

Análise Sintática



- Introdução;
- Gramática Livre de Contexto;
- Estratégias para Analisadores Sintáticos.



OBJETIVOS DE APRENDIZADO

- Apresentar todos os conceitos do analisador sintático;
- Demonstrar na prática como um analisador sintático é criado.



Orientações de estudo

Para que o conteúdo desta Disciplina seja bem aproveitado e haja maior aplicabilidade na sua formação acadêmica e atuação profissional, siga algumas recomendações básicas:



Assim:

- ✓ Organize seus estudos de maneira que passem a fazer parte da sua rotina. Por exemplo, você poderá determinar um dia e horário fixos como seu “momento do estudo”;
- ✓ Procure se alimentar e se hidratar quando for estudar; lembre-se de que uma alimentação saudável pode proporcionar melhor aproveitamento do estudo;
- ✓ No material de cada Unidade, há leituras indicadas e, entre elas, artigos científicos, livros, vídeos e *sites* para aprofundar os conhecimentos adquiridos ao longo da Unidade. Além disso, você também encontrará sugestões de conteúdo extra no item **Material Complementar**, que ampliarão sua interpretação e auxiliarão no pleno entendimento dos temas abordados;
- ✓ Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois irão auxiliar a verificar o quanto você absorveu de conhecimento, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e de aprendizagem.

Introdução

Nesta unidade, iremos estudar a análise sintática. No processo de compilação, a análise sintática é a segunda análise realizada pela fase de “Análises” de um compilador. O analisador sintático, também conhecido como *parser*, possui como objetivo principal determinar a sintaxe de um programa, isto é, determinar a estrutura de um programa. O analisador léxico obtém um conjunto de *tokens* que é passado para o analisador sintático, este que é responsável por verificar se tal conjunto possui uma estrutura sintática válida.

Em uma maneira mais simples, a análise sintática é a fase que estuda a disposição dos *tokens* em uma instrução. Por exemplo, dada a instrução; **int cont = 0;** é verificado se todos os *tokens* (elementos de uma mais simples) estão na ordem certa. Uma instrução como; **cont int 0 =;** iria gerar um erro sintático, porque os *tokens* não estão na ordem correta.

Na análise sintática, o principal mecanismo utilizado é a **Gramática Livre de Contexto (GLC)**. Uma Gramática Livre de Contexto representa uma gramática formal. As derivações de uma GLC visam determinar se o fluxo de palavras se encaixa na sintaxe da linguagem de programação.

Podemos dizer que a análise léxica encara o código-fonte como um arquivo de texto cheio de palavras que são reconhecidas através de autômatos finitos determinísticos. Em contrapartida, o analisador sintático também encara o código-fonte como um arquivo de texto. Todavia, o analisador sintático verifica se as palavras satisfazem às regras gramaticais. É por meio da gramática que é possível validar expressões criadas nas linguagens de programação.

O objetivo do analisador sintático é agrupar os *tokens* em frases gramaticais usadas pelo compilador com o **objetivo de criar uma saída que represente a estrutura hierarquia do código fonte**. O analisador sintático analisa as Regras Gramaticais de uma sentença. Regra gramatical é a forma como podemos descrever a estrutura sintática do código-fonte.

A tabela abaixo especifica entrada e saída das análises léxicas e sintáticas.

Tabela 1

Fases	Entrada	Saída
Análise Léxica	Conjunto de caracteres	Conjunto de <i>tokens</i>
Análise Sintática	Conjunto de <i>tokens</i>	Árvore sintática

Até o momento, estudamos a análise léxica e, nesta unidade, iremos estudar a análise sintática. O fluxograma apresentado na Figura 1 ilustra como essas duas análises se relacionam.

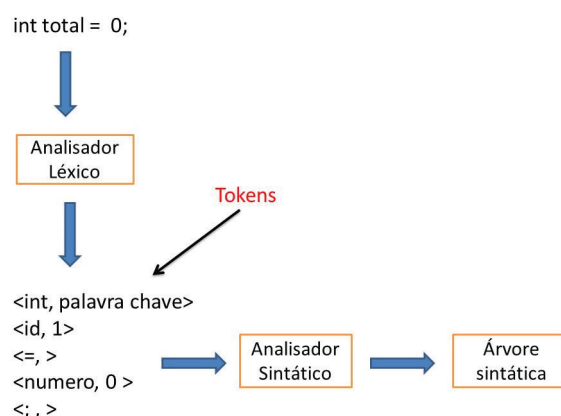


Figura 1 – Fluxograma do processo de análises
Fonte: Acervo do Conteudista

Como mencionado anteriormente, a Gramática Livre de Contexto é o principal mecanismo utilizado por um analisador sintático. GLC é abordada com mais detalhes na próxima seção.

Gramática Livre de Contexto

Uma Gramática Livre do Contexto pode ser considerada como um formalismo gerador. É uma gramática com restrições na forma das regras de produções, e tais restrições são definidas de maneira mais livre do que as gramáticas regulares (MENEZES, 2011).

As gramáticas livres de contextos têm a principal forma:

$$A \rightarrow \alpha.$$

Caso ocorra uma derivação de A, a variável A que é o contexto, não depende de qualquer símbolo que a **anteceda**, ou a **sucedá**, ou seja, é **livre de contexto**.

Uma Gramática Livre de Contexto pode ser **representada** por uma **quádrupla** $G(V, T, P, S)$ onde:

- **V** é o conjunto finito dos símbolos **não terminais**;
- **T** é o conjunto finito dos **símbolos terminais** que correspondem ao **alfabeto da linguagem** definida pela gramática;
- **P** é o conjunto das **regras de produção** da gramática;
- **S** é a **raiz da gramática (variável inicial)**.

Assim como uma gramática regular, só que com a restrição de qualquer regra de produção de P, seja da forma:

$$A \rightarrow \alpha.$$

Onde, A é uma variável de V , e α , uma palavra de $(V \cup T)^*$.

É importante observar que uma Gramática Livre de Contexto é uma gramática na qual o lado esquerdo das produções contém apenas uma variável (RAMOS, M.; VEJA, I.; JOSÉ, J., 2009). Para facilitar o entendimento, vamos observar o exemplo a seguir.

Exemplo 1. Considere a Gramática Livre de Contexto:

$$G = (\{E, T, F\}, \{a, +, *, (,)\}, P, E)$$

Onde:

$P = \{$	$E \rightarrow T + E,$	1
	$E \rightarrow T,$	2
	$T \rightarrow F * T,$	3
	$T \rightarrow F,$	4
	$F \rightarrow (E),$	5
	$F \rightarrow a \}$	6

Vamos considerar a palavra $a * (a + a)$. Essa palavra pode ser obtida a partir da sequência de derivações:

$E \xrightarrow{2} T \xrightarrow{3} F * T \xrightarrow{4} a * T \xrightarrow{4} a * F \xrightarrow{5} a * (E) \xrightarrow{1} a * (T + E) \xrightarrow{1} a * (F + E) \xrightarrow{6} a * (a + E) \xrightarrow{2} a * (a + T) \xrightarrow{4} a * (a + F) \xrightarrow{6} a * (a + a)$

Essa sequência de derivações corresponde às aplicações das regras de produções 2, 3, 6, 4, 5, 1, 4, 6, 2, 4, 6 respectivamente.

A linguagem gerada por essa gramática (gramática do exemplo) compreende as palavras que representam expressões aritméticas corretamente formadas sobre o operando a com operadores “+” e “*”. Essa gramática permite também, gerar **subexpressões** delimitadas através de parênteses. Essas subexpressões podem ser geradas com base nas mesmas regras utilizadas para construir a expressão inicial (RAMOS, M.; VEJA, I.; JOSÉ, J., 2009).

Analísadores sintáticos utilizam como mecanismo principal as GLC. Devemos observar que as derivações realizadas pela GLC possuem como objetivo determinar se um fluxo de palavras se encaixa na sintaxe de linguagem de programação desenvolvida.

Notação BNF

Uma maneira usual de representar uma gramática livre de contexto é o uso da Forma **Backus Naur Form**, ou simplesmente **BNF**. Em uma BNF, vale que:

- As **variáveis** são palavras delimitadas pelos símbolos $\langle T \rangle$.
- As palavras **não delimitadas são terminais**.

- A regra de produção $A \rightarrow a$ é substituído por:

$$A ::= a$$

- O $|$ é usado para separar os símbolos.

Para facilitar o entendimento, vamos observar os exemplos a seguir:

Exemplo 1. Dadas as regras de produções a seguir:

$$F \rightarrow a$$

$$F \rightarrow b$$

Temos as seguintes BNF:

$$\langle F \rangle ::= a$$

$$\langle F \rangle ::= b$$

Ou ainda,

$$\langle F \rangle ::= a \mid b$$

Agora, vamos considerar um exemplo dentro de um contexto claro e objetivo.

Exemplo 2: Vamos supor que seja necessário definir uma BNF capaz de gerar qualquer identificador válido na linguagem de programação JAVA (MENEZES, 2011). Assim:

- Toda letra é um identificador;
- Se S é um identificador, então a concatenação à direita de S com qualquer letra ou dígito também é um identificador.

Assim, temos uma BNF como se segue:

$$\langle \text{identificador} \rangle ::= \langle \text{letra} \rangle \mid \langle \text{identificador} \rangle \langle \text{letra} \rangle \mid \langle \text{identificador} \rangle \langle \text{dígito} \rangle$$

$$\langle \text{letra} \rangle ::= a \mid b \mid z$$

$$\langle \text{dígito} \rangle ::= 0 \mid 1 \mid \dots \mid 9$$

Árvore de Derivação

Algumas vezes, pode ser útil representar a **estrutura** de **sentenças** ou formas sentenciais de uma linguagem livre de contexto. Habitualmente, essa representação é realizada através de uma **árvore de derivação**, chamada também de **árvore sintática**. Uma árvore de derivação é muito útil para representar as estruturas sentenciais uma vez que:

- viabiliza meios para uma melhor visualização da estrutura das sentenças das linguagens, facilitando a análise das mesmas;
- ajuda na demonstração formal de teoremas, na interpretação de resultados e na assimilação de vários conceitos;
- simplifica a representação interna nos compiladores e interpretadores.

Em suma, uma árvore de derivação impõe **estruturas hierárquicas às sentenças das linguagens geradas** (RAMOS, M.; VEJA, I.; JOSÉ, J., 2009). Uma definição formal de Árvore de derivação pode ser dada como; um sistema de representação de sequências de derivações em uma gramática livre de contexto $G(V, T, P, S)$, onde vale que:

- a **raiz é o símbolo inicial** da gramática;
- os **vértices interiores** (ou nós internos) são **variáveis**. Se A é um vértice interior e $X_1, X_2, \dots, X_n$ são “filhos” de A , então:
 - » $A \rightarrow X_1, X_2, \dots, X_n$ é uma produção da gramática;
 - » Os vértices $X_1, X_2, \dots, X_n$ são ordenados da esquerda para a direita;
- os **vértices folhas** (ou nós folha) são símbolos terminais ou símbolo vazios. O vazio é o único filho de seu pai ($A \rightarrow \epsilon$).

A Figura 2 ilustra os elementos de uma árvore de derivação.

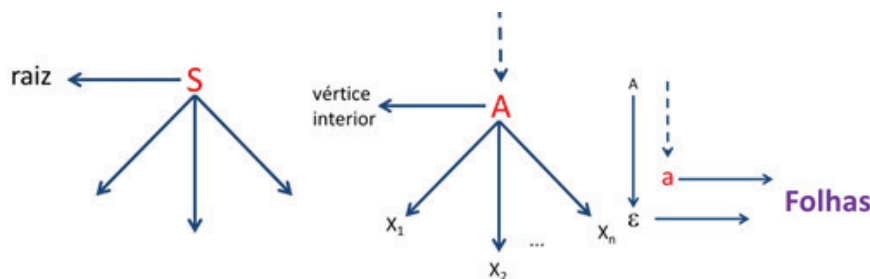


Figura 2 – Elementos da Árvore de Derivação

Fonte: Acervo do Conteudista

Exemplo 1. Dada a GLC apresentada anteriormente, vamos criar a árvore de derivação para a palavra a^* ($a + a$). Assim:

Considere a Gramática Livre de Contexto

$$G = (\{E, T, F\}, \{a, +, *, (,)\}, P, E)$$

Onde:

$P = \{$	$E \rightarrow T + E,$	1
	$E \rightarrow T,$	2
	$T \rightarrow F * T,$	3
	$T \rightarrow F,$	4

$F \rightarrow (E),$

5

$F \rightarrow a \}$

6

Derivando a palavra **a * (a + a)**, obtemos a seguinte árvore de derivação:

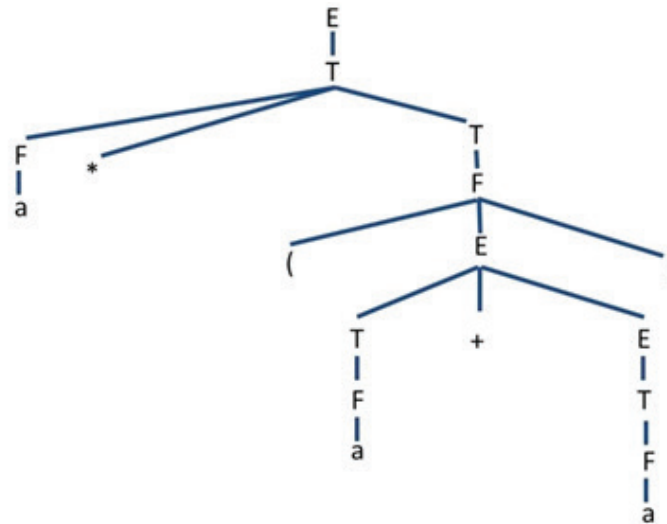
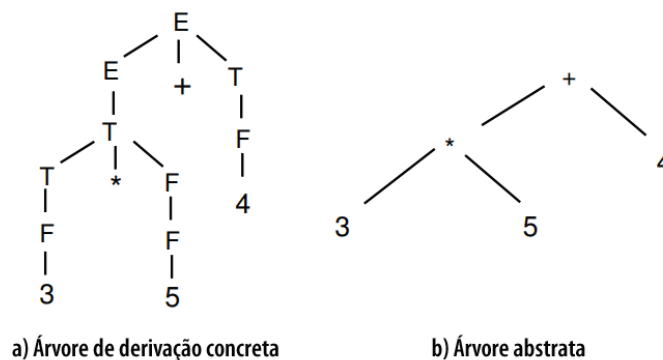


Figura 3 – Árvore de derivação para a palavra a * (a + a)

Fonte: Adaptado de Ramos, M.; Veja, I.; José, J., 2009

Árvore Abstrata

Uma árvore abstrata é uma árvore gramatical na qual os operadores e palavras-chave não aparecem como folhas, mas como nós interiores da árvore. Nessa árvore, só aparece o essencial.



a) Árvore de derivação concreta

b) Árvore abstrata

Figura 4 – Árvore Abstrata

Fonte: Acervo do Conteudista

Gramática Ambígua

A partir de uma **mesma palavra**, algumas vezes, pode-se obter **mais de uma árvore** devido à escolha do **lado da derivação (esquerdo ou direito)**. Uma derivação mais à **esquerda** de uma palavra, sempre obtém o símbolo não terminal mais à esquerda. Já uma derivação mais à **direita** de uma palavra sempre obtém o símbolo

não terminal mais à direita. Caso seja possível, numa mesma palavra, obter duas ou mais árvores de derivação, tem-se caracterizado uma **Gramática Ambígua**.

Exemplo: Considere a GLC a seguir:

$$G = (\{E\}, \{a, +, -\}, P, E)$$

onde;

- | | | |
|----------|------------------------|---|
| $P = \{$ | $E \rightarrow T + E,$ | 1 |
| | $E \rightarrow E - E,$ | 2 |
| | $E \rightarrow a \}$ | 3 |

Agora vamos criar a árvore de derivação para a palavra **a + a - a**. Se começamos a derivar a partir da regra 1, temos a árvore de derivação apresentada na Figura 5.a. Agora, se escolhemos a regra 2 no começo da derivação, teremos a árvore apresentada na Figura 5.b. Existe mais de uma árvore para a GLC acima. Dessa forma, é possível concluir que essa GLC é ambígua.

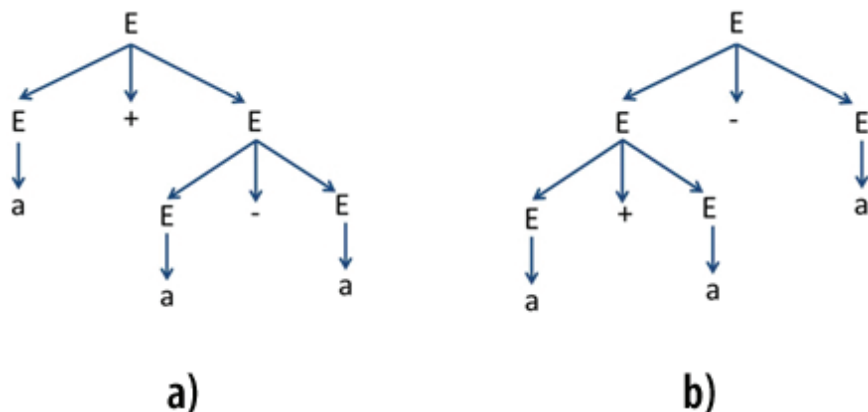


Figura 5 – Árvore de derivação para a palavra a + (a - a)

Fonte: Acervo do Conteudista

Estratégias para Analisadores Sintáticos

Habitualmente, a sintaxe de uma linguagem de programação é normalmente dada pelas **regras gramaticais** de uma **Gramática Livre de Contexto**. Existem diversos algoritmos que implementam Gramática Livre de Contexto para construir analisadores sintáticos. Uma GLC produz uma árvore de derivação, certo? Há duas classes categorias gerais de algoritmos para análise sintática; **Top-Down (Descendente)** e **Bottom-up (ascendente)**. A principal diferença entre as duas é a maneira como a árvore é construída. Na análise **descendente**, a árvore é construída da raiz até as folhas. Na análise **ascendente**, a árvore é construída das folhas até a raiz.

Análise Sintática Descendente

Um algoritmo para análise sintática **descendente** analisa a cadeia de elementos de entrada pelo acompanhamento dos passos de uma derivação à esquerda. O nome “**descendente**” vem da maneira como a árvore de **análise sintática é percorrida, da raiz para as folhas**. Na classe de algoritmos descendente, há duas formas de analisadores sintáticos: **analisadores preditivos** e **analisadores com retrocesso (*backtracking*)**. Um **analisador sintático preditivo** tenta prever a construção seguinte da cadeia entrada com base em um ou mais elementos de verificação à frente. Agora, um analisador sintático com retrocesso testa diferentes possibilidades de análise sintática da entrada, retrocedendo se alguma possibilidade falhar. Os analisadores com retrocesso são mais lentos e requerem tempo exponencial para a execução. Dessa forma, os analisadores com retrocesso não são muito utilizados em compiladores, por causa do seu desempenho.

Na análise **descendente**, a árvore é construída utilizando **derivações**. Nesse tipo de análise sintática, temos os métodos de análises **descendentes recursivos** e os **LL(1)**. A análise sintática descendente recursiva é bastante versátil e é mais adequada para um analisador sintático escrito manualmente. Já o método **LL(1)** tenta deduzir as produções da gramática a partir do nó raiz. O nome **LL(1)** provem dos seguintes fatos: o primeiro “L” se refere ao fato de que esse método lê a entrada da esquerda para direito, isto é, o processamento da entrada ocorre da esquerda para a direita. O segundo “L” se refere ao fato de o analisador acompanhar uma derivação à esquerda para a entrada, isto é, produz uma derivação mais à esquerda. Agora o número 1 entre parênteses significa que esse método usa apenas um símbolo da entrada para prever a direção da análise. Também existe o método LL(k) que utiliza k símbolos para prever a direção da análise, todavia, apenas um símbolo é mais comum.

Na análise **sintática LL(1)**, é utilizada uma pilha explícita. A representação dessa pilha é de forma padrão e é útil para facilitar e agilizar a visualização das ações do analisador sintático LL(1). Para explicar o algoritmo do LL(1), vamos considerar uma gramática muito simples que gera cadeias de parênteses balanceados.

$$S (S) S \mid \varepsilon$$

A tabela a seguir mostra ações de um analisador sintático descendente LL(1) para a gramática e a cadeia (). Esta tabela possui três colunas, a primeira mostra o conteúdo da pilha de análise sintática, com o final da pilha à esquerda e o topo da pilha à direita. O final da pilha é indicado com um cifrão.

Tabela 2

Pilha de análise sintática	Entrada	Ação
\$ S	() \$	$S \rightarrow (S) S$
\$ S) S (	() \$	casamento
\$ S) S	) \$	$S \rightarrow \varepsilon$

Pilha de análise sintática	Entrada	Ação
\$ S)	) \$	Casamento
\$ S		$S \rightarrow \epsilon$
\$ S		Aceita

Pelo fato do LL(1) ser uma estratégia (algoritmo) para analisadores sintáticos descendente, a pilha começa sempre com a raiz, porque, na análise descendente, a árvore é constituída da raiz até as folhas. Por isso a primeira linha da tabela contém:

\$ S

cifrão para indicar o final da pilha e o S que é a raiz. A segunda coluna contém a entrada que será processada. Os símbolos da entrada são processados da esquerda para a direita. Um cifrão marca o final da entrada. A terceira coluna é a descrição resumida da ação do analisador sintático que altera a pilha e a entrada, conforme mostra a segunda linha da tabela,) \$.

Um analisador sintático descendente LL(1) **aceita uma cadeia de entrada se, após diversas ações, a pilha e a entrada ficarem vazias.** Portanto, um esquema geral do LL(1) bem sucedido é:

\$	SímboloInicial	CadeiaEntrada	\$
			
			
\$			\$aceita

Para exemplificar o LL(1), estamos considerando a *encontra* como () e a raiz como S. A ideia geral do LL(1) é substituir um não terminal do topo da pilha por uma de suas escolhas na regra gramatical. Isso é feito para produzir o elemento corrente de entrada no topo da pilha, caso tenha casado com o símbolo de entrada e possa descartá-la tanto da pilha como da entrada. Ações que podem ocorrer:

- substituir um não terminal A no topo da pilha por uma cadeia α com base na escolha da regra gramatical $A \rightarrow \alpha$;
- casar uma marca no topo da pilha com a marca de entrada seguinte.

Um analisador LL(1) pode tomar essas duas ações. A ação **a** pode ser denominada como **geradora**. A ação **b** casa um símbolo de entrada com um símbolo terminal alocado no topo da pilha; quando isso ocorre, dizemos que ocorreu um **casamento** e tanto o símbolo de entrada, como o símbolo terminal que está no topo são descartados. Em línguas gerais, se houver um símbolo não terminal no topo da pilha, é aplicada uma regra gramatical do tipo $A \rightarrow \alpha$. Nesse processo, A é substituído por α na pilha. É importante observar que, nessa ação, a substituição α precisa ser colocada invertida na pilha (isso garante que a cadeia α virá no topo da pilha ordenada da esquerda para a direita).

Por exemplo, no passo 1 da análise sintática da tabela acima, a pilha e a entrada são:

$$\$ S () \$$$

e a regra que utilizamos para substituir S no topo da pilha é $S \rightarrow (S) S$, assim, a cadeia $S) S ($ é colocada na pilha, para obter na pilha

$$\$ S) S (() \$$$

agora, no topo da pilha há um terminal. Toda vez em que há um terminal no topo da pilha, é necessário verificar se esse símbolo terminal é igual ao símbolo da entrada; se for, ótimo! Há um **casamento**. Quando há um casamento, o símbolo do topo da pilha e o símbolo da entrada são descartados. Assim, temos:

$$\$ S) S) \$$$

Agora temos um terminal no símbolo não terminal no topo, devemos aplicar uma regra gramatical, assim, aplicamos a regra $S \rightarrow \epsilon$. Obtemos o seguinte:

$$\$ S) \quad) \$$$

o S foi substituído por vazio, ϵ . Agora temos um símbolo terminal no topo que bate com o símbolo de entrada, assim, temos um **casamento** e obtemos:

$$\$ S \quad \$$$

Agora aplicamos a regra $S \rightarrow \epsilon$ e obtemos:

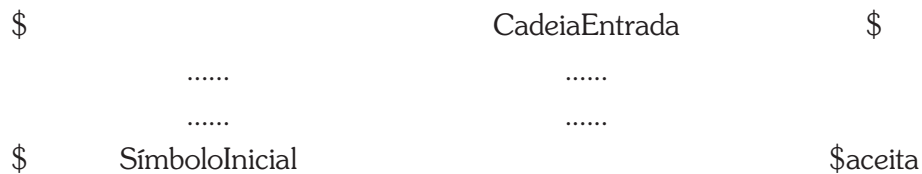
$$\$ \quad \$$$

A pilha e a entrada terminaram vazias, dessa forma, a entrada $()$ é aceita pela gramática em questão.

Análise Sintática Ascendente

Na análise **descendente**, a árvore é construída da raiz até as folhas. Nessa análise, o algoritmo mais utilizado é o **LR(1)**. O nome desse algoritmo provem dos seguintes fatos: o L indica que a entrada é processada da esquerda para a direita, enquanto que o R indica que a derivação à direita é produzida e o número 1 indica que um símbolo de verificação à frente é utilizado. Há variações do algoritmo LR(1) como o LR(0), que, sem verificar nenhum símbolo à frente, consegue tomar decisões durante a análise. Uma versão melhorada da análise **LR(0)**, que faz uso da verificação à frente é denominada análise sintática **SLR(1)**. Nesse algoritmo, o S indica análise sintática LR(1) simples. Outro método um pouco mais poderoso do que a análise **SLR(1)**, mas menos complexo que a análise LR(1), é denominado análise sintática **LALR(1)**, LA indica verificação à frente em análise LR(1).

Em um primeiro momento, vamos focar no algoritmo LR(1). Um analisador sintático ascendente LR(1) usa uma pilha explicitamente para efetuar a análise, de maneira similar a um analisador descendente. Na análise ascendente, a árvore é constituída das folhas até a raiz. Assim, a pilha está vazia no início da análise ascendente e conterá o símbolo inicial ao final de uma análise bem-sucedida. Um esquema geral da análise sintática ascendente pode ser dada como:



onde a pilha da análise sintática fica à esquerda, a entrada no centro e as ações do analisador à direita. Um analisador ascendente tem duas ações possíveis:

- Carrega um terminal do topo da entrada para o topo da pilha;
- Reduz uma cadeia α do topo da pilha para um não terminal A, dada a escolha $A \rightarrow \alpha$.

Assim, um analisador ascendente também é conhecido como analisador **carrega-reduz**, ou **shift-reduce** em inglês. O **carregar** é indicado pela palavra **carrega**. O **reduzir** é indicado pela palavra **reduz** junto com a escolha da regra gramatical. Na análise **descendente**, a árvore é construída utilizando **derivações** $A \rightarrow \alpha$. Na análise ascendente, a árvore é construída através de **Reduções** $\alpha \rightarrow A$. A redução é o inverso da derivação. Podemos mencionar, ainda, que a **análise ascendente** produz uma **derivação mais à direita**. Enquanto a **análise descendente** produz uma **derivação mais à esquerda**.

Uma característica adicional dos analisadores sintáticos ascendentes é que, por motivos técnicos, a gramática é sempre aumentada com um novo símbolo inicial. Isso significa que, se S for o símbolo inicial, um novo símbolo inicial S' é acrescentado à gramática, com uma única produção unitária para o símbolo inicial anterior:

$$S' \rightarrow S$$

Para exemplificar o LR(1), vamos considerar a gramática para balancear parênteses:

$$S' \rightarrow S$$

$$S \rightarrow (S) S \mid \varepsilon$$

A tabela a seguir apresenta uma análise ascendente para a cadeia () com base na gramática acima:

Tabela 3

Pilha de análise sintática	Entrada	Ação
\$	()\$	Carrega
\$(	)\$	reduz $S \rightarrow \epsilon$
\$(S	)\$	Carrega
\$(S)	\$	reduz $S \rightarrow \epsilon$
\$(S)S	\$	reduz $S \rightarrow (S)S$
\$S	\$	reduz $S' \rightarrow S$
\$S'	\$	Aceita

Na análise ascendente, a árvore é gerada das folhas até a raiz. O algoritmo LR(1) é um algoritmo de análise sintática ascendente. Assim, **a pilha começa vazia (folhas) e termina com o símbolo inicial (raiz)**. O algoritmo LR(1) só pode fazer duas ações, **carregar** e **reduzir**. Carregar significa deslocar um símbolo da entrada para o topo da pilha. Já Reduzir significa utilizar alguma regra gramática para reduzir a expressão. Podemos escolher qual ação devemos utilizar, carregar ou reduzir. Na primeira linha da tabela, decidimos carregar. A primeira linha se encontra da seguinte forma:

\$ ()\$ carrega

como decidimos carregar, então a segunda linha da tabela fica como a seguir:

\$()\$ Reduz $S \rightarrow \epsilon$

agora, decidimos utilizar a regra Reduz $S \rightarrow \epsilon$ para reduzir. Assim, a terceira linha fica:

\$(S)\$ carrega

Vale ressaltar que, pela regra $S \rightarrow \epsilon$, saímos de vazio para S. Por isso o topo da pilha passa a conter S. Agora, a ação é carregar, então, a linha seguinte fica:

\$(S) \$ Reduz $S \rightarrow \epsilon$

agora, realizamos uma operação de Reduz e saímos de vazio ϵ e chegamos em S

\$(S)S \$ Reduz $S \rightarrow \epsilon$

e realizamos outra operação de reduz, agora utilizamos a regra $S \rightarrow (S)S$ e temos:

\$S \$ $S' \rightarrow S$

para finalizar, iremos realizar mais uma operação de reduz $S' \rightarrow S$, agora temos:

\$S' \$ Aceita

Chegamos à raiz e a entrada está vazia, assim, a entrada é aceita pela gramática.

Segundo Louden (2004), um analisador ascendente pode carregar os símbolos de entrada para a pilha até determinar qual ação deve executar. Durante o processamento, quando o analisador não consegue decidir se deve carregar ou reduzir, temos um **erro carregar/reduzir**. Agora, quando o analisador não consegue determinar qual redução fazer, temos um erro **Reduz/ Reduz**.

Handle

Neste momento, você deve estar se perguntando, “Como o analisador sintático ascendente escolhe qual ação correta tomar?” ou, ainda, “Qual regra de produção o analisador sintático deve escolher?”. Bom... vamos lá! Antes, vale ressaltar que uma escolha errada pode levar a um erro fatal, mesmo para uma entrada válida. Nós, seres humanos, escolhemos qual regra utilizar por meio da **intuição**, isto é, sempre devemos utilizar uma regra que sempre nos permita realizar uma redução que nos leva para uma situação de sucesso. Em outras palavras, devemos reduzir sempre que encontramos um **Handle**. Todavia, o que é um **Handle**? **Handle** é uma sequência de símbolos do lado direito da produção, que permita realizar uma redução. Assim, sempre queremos reduzir quando o topo da pilha for um *handle*. Não existe um algoritmo infalível e eficiente para reconhecer um *handle* no topo da pilha *shift-reduce*. A maioria reconhece (ou não) um *handle* examinando a pilha e o próximo *token* da entrada (o *lookahead*).

Lookaheads

Lookaheads permite olhar o próximo *token* que será verificado e decidi qual regra gramatical deve ser usada. Por exemplo, ao analisar o *token* “/” , é necessário observar qual é o próximo *token*, antes de decidir qual regra usar, porque o próximo *token* pode ser outro “/” ou “*”, comentando uma linha ou um bloco de instruções, “//” ou “/*”.



Em Síntese

O foco desta unidade é análise sintática, que é a segunda análise realizada no processo de compilação. A análise sintática é responsável por verificar a estrutura de uma instrução. Por exemplo, se todos os seus elementos estão na ordem correta.

Na análise sintática, dada uma gramática *G* e uma sentença *X*, o objetivo do analisador sintático é verificar se *X* está sintaticamente correta perante a linguagem *G* e se realmente existe na linguagem *G*. Dada a sentença *X*, o analisador léxico obtém uma sequência de tokens, que é passado para o analisador sintático, este, produz uma árvore de derivação caso a sentença *X* seja válida, ou gera um erro se a sentença é inválida.

Para implementar um analisador sintático devemos usar uma Gramática Livre de Contexto (GLC). Com uma GLC é possível criar uma árvore de derivação que permite impor uma hierarquia nas instruções e observar a sua estrutura. Para implementar um analisador sintático, pode-se utilizar análise ascendente ou descendente. Na análise ascendente é criada uma árvore das folhas até a raiz. Já na análise descendente é criada uma árvore da raiz até as folhas. .

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:



Livros

Compiladores – Da Teoria à Prática

SANTOS, P. R.; LANGLOIS, T. **Compiladores – Da Teoria à Prática**. LTC, Rio de Janeiro 2018

Optimizing compilers for modern architectures: a dependence-based approach

ALLEN, R.; KENNEDY, K. ***Optimizing compilers for modern architectures: a dependence-based approach***. Taylor & Francis US, 2002.



Vídeos

Linguagens e Compiladores – Conceitos de Análise Sintática e Análise Sintática Descendente

<https://youtu.be/JQ92w5oDyb0>



Leitura

Compiler Construction

WAITE, W. M.; GOOS, G. ***Compiler construction***. Springer Science & Business Media, 2012.

<http://bit.ly/2PqDxxy>

Referências

AHO, A. V. **Compiladores**: Princípios, Técnicas e Ferramentas. 2. ed. Pearson. (e-book)

APPEL, A. W. *Modern Compiler Implementation In: **Java***. 2. ed. New York: Cambridge University Press, 2002.

BERGMANN, S. D. **Compiler Design: Theory, Tools and Examples**, 2016. <<http://www.cs.cmu.edu/~aplatzer/course/Compilers/waitegoos.pdf>>. (e-book)

LOUDEN, K. C. **Compiladores**: Princípios e Práticas. São Paulo: Pioneira Thomson Learning, 2004.

MENEZES, P. B. **Linguagens formais e autômatos**. 6. ed. São Paulo: Bookman, 2011.

PRICE, A. M. A. **Implementação de Linguagens de Programação**: Compiladores. 2. ed. Porto Alegre: Sagra Luzzatto, 2001.



Cruzeiro do Sul
Educatonal